Масштабирование

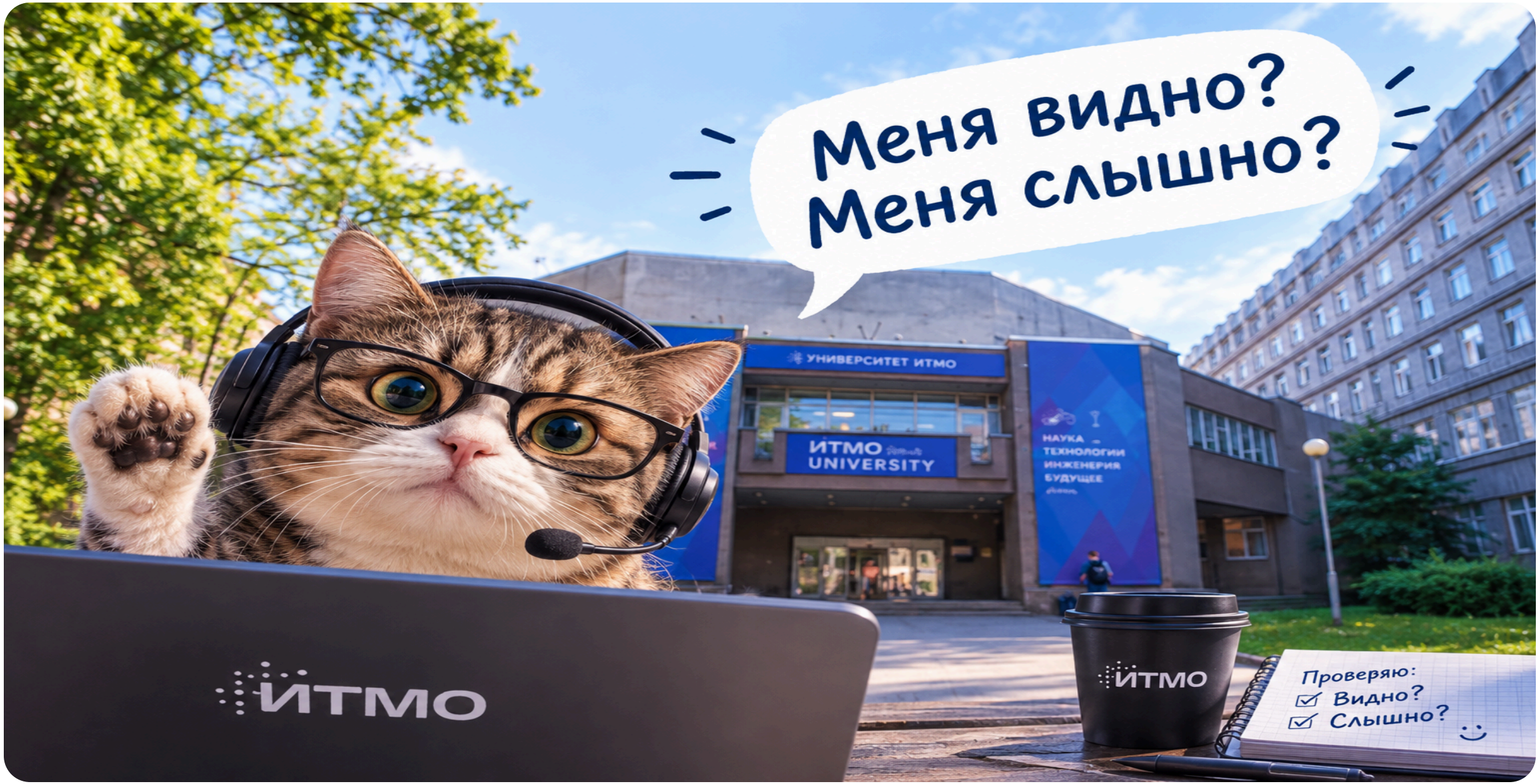
Семён Мартынов, Кирилл Дикалин, Георгий Семёнов

ИНСТИТУТ ПРИКЛАДНЫХ КОМПЬЮТЕРНЫХ НАУК

Национальный исследовательский университет ИТМО

15 октября 2026

Проверка связи



Цели лекции

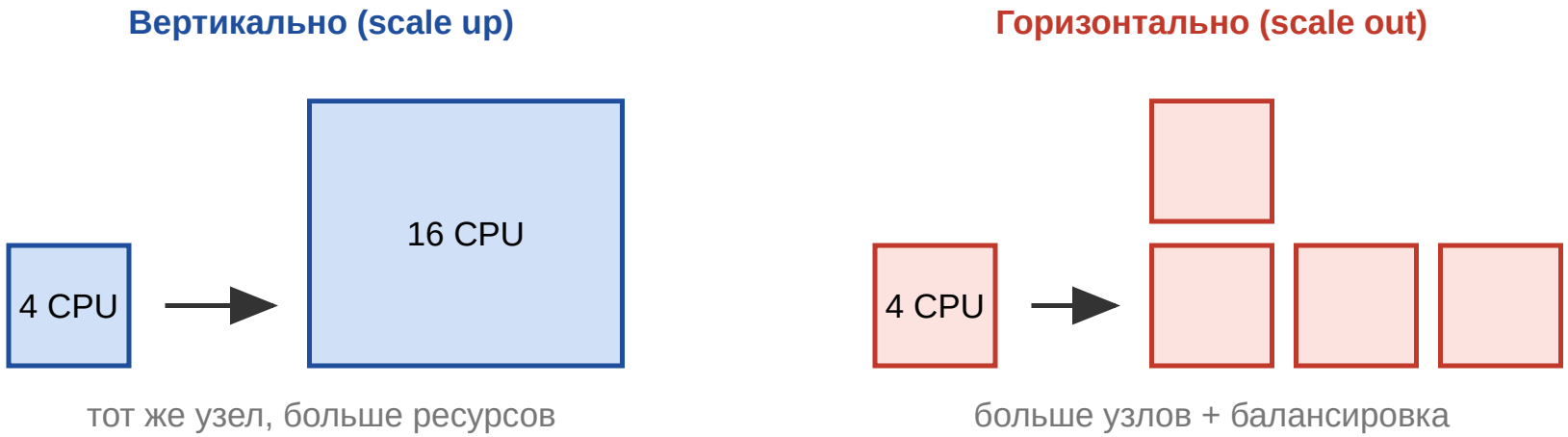
1. **Различать** вертикальное и горизонтальное масштабирование и **называть** пределы каждого.
2. **Объяснять**, чем сервисы с состоянием отличаются от сервисов без состояния и почему это определяет способ масштабирования.
3. **Оценивать** потолок ускорения по закону Амдала и по универсальному закону масштабируемости (USL).
4. **Находить** узкое место и **предсказывать**, куда оно сместится после масштабирования.
5. **Считать** число экземпляров с учётом запаса, пика и потери части мощности.

Вопрос лекции:
почему вдвое больше серверов почти никогда не даёт вдвое больше производительности?

План лекции

1. Виды масштабирования: scale up, scale out, stateless и stateful
2. Пределы ускорения: закон Амдала, USL, contention и coherency
3. Узкие места: поиск, смещение после масштабирования, горячий ресурс
4. Планирование мощности: запас, пик, потеря части мощности, число экземпляров

3.1 Вертикальное и горизонтальное масштабирование



- **Вертикальное** (scale up): мощнее узел — больше ядер, памяти, быстрее диск и сеть
- **Горизонтальное** (scale out): больше узлов и распределение работы между ними
- Реальные системы **комбинируют** оба подхода: сначала вертикально, пока дёшево и просто, затем горизонтально

3.1 Что выбрать: сравнение

	Вертикальное	Горизонтальное
Сложность	**низкая**: приложение не меняется	**выше**: балансировка, распределение данных, частичные отказы
Предел	потолок самой мощной машины; у топовых конфигураций цена нередко растёт быстрее мощности	жёсткого потолка по железу нет, но действуют законы 3.2: отдача падает, а затем может стать отрицательной
Отказ	единая точка отказа	отказ одного узла — потеря **доли** мощности
Простой при росте	часто нужен рестарт или смена типа машины	узлы добавляются без остановки сервиса
Что требуется от кода	ничего, если код умеет использовать ядра	сервис должен **допускать** несколько экземпляров

Горизонтальное масштабирование — свойство **архитектуры**, а не только инфраструктуры: добавить узлы можно всегда, а получить от них пользу — нет.

3.1 Сервисы без состояния и с состоянием

	Stateless	Stateful
Состояние запроса	целиком в запросе или во внешнем хранилище	хранится **в экземпляре**
Любой экземпляр может обслужить запрос	**да**	нет: нужен «свой» экземпляр
Добавление экземпляра	запустить и включить в балансировщик	**перенести данные** (resharding, rebalancing)
Отказ экземпляра	запрос повторяется на другом	потеря данных или недоступность, если нет реплики
Примеры	HTTP-обработчики, воркеры	БД, кэши, брокеры, сессии в памяти

- Состояние **не исчезает**, а переезжает в хранилище: stateless-слой масштабируется легко, потому что трудности сдвинуты в **stateful-слой** под ним
- Чем больше состояния в экземпляре, тем дороже его добавлять, убирать и заменять

3.1 Масштабируем компоненты по отдельности

Компонент	Как масштабируется	Цена
Балансировщик	несколько экземпляров, DNS или anycast	сам может стать узким местом
Приложение (stateless)	больше экземпляров	нагрузка переходит вниз по цепочке
Кэш	шардирование по ключу	кэш-промахи после смены топологии
БД: чтения	реплики	отставание реплик (replication lag)
БД: записи	шардирование	межшардовые операции и транзакции дороги
Очередь / брокер	партиции	порядок гарантируется только внутри партиции

- У каждого компонента **свой** профиль нагрузки и **свой** способ масштабирования
- Масштабировать всю систему «в целом» бессмысленно: растёт тот компонент, который упирается в предел (см. 3.3)

3.2 Закон Амдала

Доля работы p ускоряется в N раз, остальное $(1-p)$ — строго последовательно:

$$\boxed{S(N) = \frac{1}{(1-p) + \dfrac{p}{N}}}, \qquad S(\infty) = \frac{1}{1-p}$$

N	2	8	64	1024	∞
$S(N)$ при $p = 0.95$	1.90	5.93	15.42	19.64	**20**

- Последовательная доля 5% ограничивает ускорение **двадцатью**, сколько бы ресурсов ни добавили
- Последовательная часть — это не только код: блокировка, единая БД, один лидер, общая очередь

Идея: G. M. Amdahl, AFIPS Spring Joint Computer Conference, 1967. В оригинальной статье формулы нет, её выписали позже.

3.2 Универсальный закон масштабируемости

$$C(N) = \frac{N}{1 + \alpha\,(N-1) + \beta\,N\,(N-1)}$$

$$N^* = \sqrt{\frac{1-\alpha}{\beta}}$$

- α — **конкуренция** (contention): ожидание общего ресурса, последовательная часть
- β — **согласование** (coherency): узлы обмениваются состоянием «все со всеми», число пар растёт **квадратично**, $\sim N(N-1)$
- При $\beta = 0$ получается закон Амдала; при $\beta > 0$ после N^* производительность **снижается**, а не просто перестаёт расти

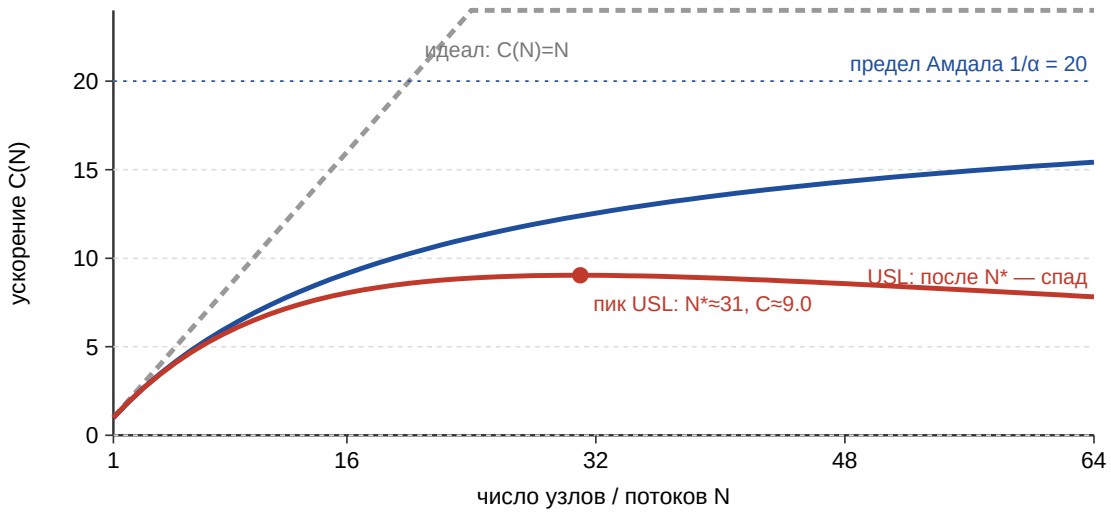


Иллюстрация: $\alpha = 0.05$, $\beta = 0.001$.

N. J. Gunther, Guerrilla Capacity Planning, 2007.

3.2 Откуда берутся α и β

	Конкуренция α	Согласование β
Суть	очередь к общему ресурсу	обмен состоянием между узлами
Примеры	глобальный lock, один лидер (в том числе в Raft), синхронная репликация через лидера, одна БД, общий пул соединений	взаимные инвалидации кэшей, когерентность кэшей процессора, протоколы «все со всеми» (например, BFT-консенсус)
Как растёт цена	доля ожидания **постоянна**	пропорционально **числу пар** узлов
Что делать	убирать общий ресурс, дробить, шардировать	реже синхронизироваться, ослаблять согласованность

3.2 Подбор α и β по замерам

- Замеряем пропускную способность $X(N)$ при нескольких N, **включая** $N = 1$, и нормируем: $C(N) = X(N)/X(1)$
- Подбираем α и β регрессией, затем проверяем прогноз на **ещё одном** N, не входившем в подгонку
- Замеры нужны **вблизи ожидаемого пика**: по точкам до 16 узлов пик при $N^* \approx 31$ — это экстраполяция, поэтому оцениваем доверительные интервалы α и β
- Отрицательные или статистически незначимые оценки — сигнал перепроверить измерения и модель: суперлинейность (например, из-за кэшей) или смена ограничивающего ресурса

Модель описывает **форму** кривой, а не причину. Ищите источник α и β профилированием, а не подгонкой.

3.2 Ограничения моделей

- Амдал предполагает **фиксированный размер задачи**. Если с ростом ресурсов растёт и задача, ускорение масштабируется иначе (*Gustafson, 1988*)
- Для сервисов часто важна не скорость одного запроса, а **пропускная способность** при ограничении на задержку: масштабирование по Густафсону здесь ближе к реальности
- Ускорение — не то же, что **эффективность**: распределённая система может быть медленнее хорошей однопоточной (*McSherry, Isard, Murray, «Scalability! But at what COST?», HotOS 2015*)
- Сначала **оптимизируем и измеряем один узел**, затем масштабируем: иначе умножаем неэффективность на N

Амдал при фиксированном размере задачи оценивает ускорение сверху, USL — описательная модель, подгоняемая по замерам. Ни то ни другое не гарантия: бывают и суперлинейные участки (например, из-за кэшей). Реальную кривую даёт эксперимент.

3.3 Узкое место

Узкое место (bottleneck) — компонент, ёмкость которого ограничивает пропускную способность всей цепочки.

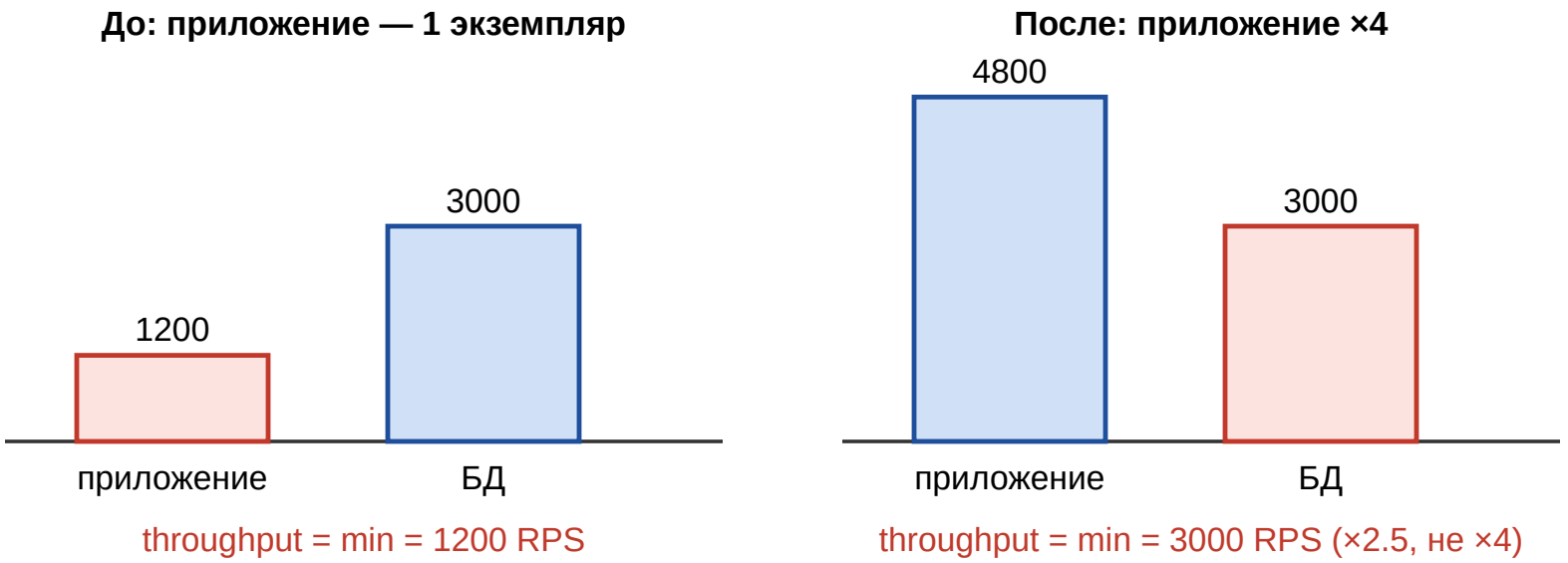
Для последовательных стадий: $X_{max} = \min_i \frac{\mu_i}{V_i}$, где μ_i — ёмкость стадии i (обращений в секунду), а V_i — число обращений к ней на **один** запрос.

Признаки узкого места (метод USE из лекции 1 и задержка):

Признак	Что смотреть
Высокая **загрузка**	CPU, диск, сеть, пул соединений близки к 100%
Есть **насыщение**	растут очереди, run queue, pool wait, consumer lag
Растут **ошибки и таймауты**	отказы там, где ресурс исчерпан
Рост **задержки** (не входит в USE)	ожидание блокировок, внешних вызовов при невысокой загрузке ресурсов

Правило:
ускорять нужно **самый загруженный** компонент; ускорение остальных не меняет X_{max}.

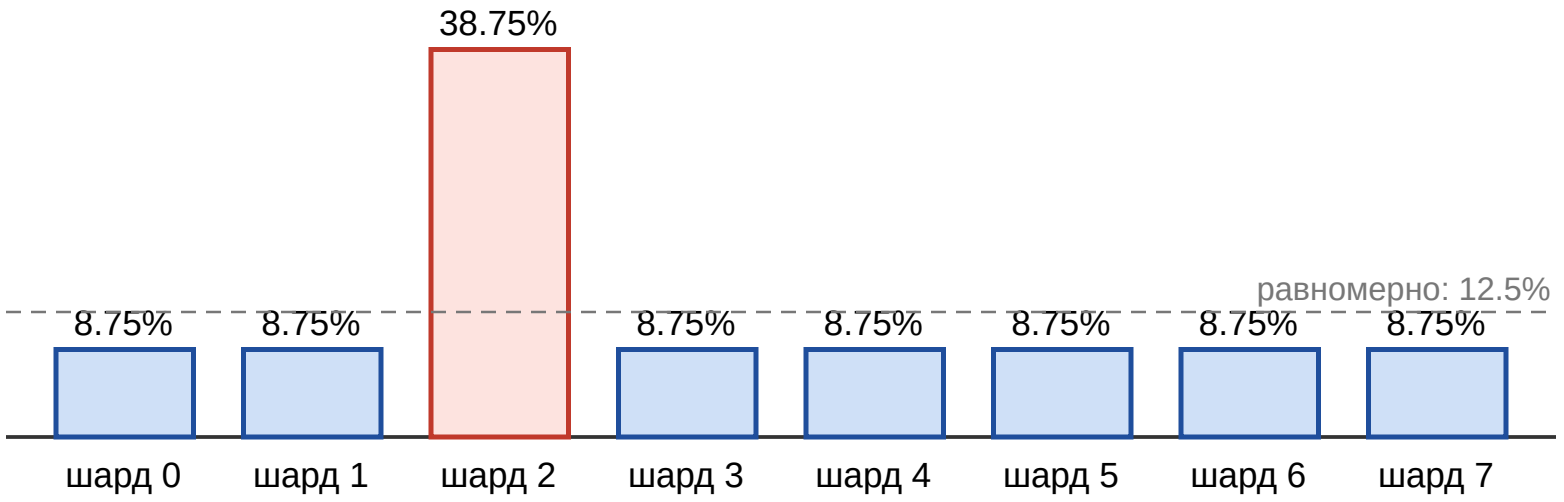
3.3 Узкое место смещается



ёмкость компонента, RPS (красный — узкое место)

- Масштабирование снимает **одно** ограничение и обнажает **следующее**: ниже по цепочке растёт нагрузка
- На диаграмме $V_{\text{БД}} = 1$. Если запрос делает 3 обращения к БД, её предел для системы — не 3000, а $3000/3 = 1000$ RPS: считать нужно **на запрос**, а не по сырой ёмкости
- Типовая цепочка смещения: CPU приложения → соединения с БД → записи в БД → сеть → внешний сервис
- После каждого изменения заново **измеряем**: пройдите цикл «нагрузка → измерение → узкое место → изменение»

3.3 Горячий ресурс и неравномерная нагрузка



доля трафика на шард: один ключ даёт 30% запросов

Шардов 8, один ключ даёт 30% запросов, остальные 70% распределены равномерно: горячий шард получает $0.3 + 0.7/8 = 38.75\%$ трафика, то есть **в 3.1 раза больше среднего**.

Кластер насыщается, когда насыщается горячий шард:
$X_{max} = \mu_{shard}/0.3875 \approx 2.58\,\mu_{shard}$ вместо $8\,\mu_{shard}$, **32%** от идеала.

3.3 Что делать с горячим ресурсом

Причина	Приём
Один «звёздный» ключ	кэш перед шардом с коротким TTL; разбиение ключа на несколько (key splitting); отдельные ресурсы для самых горячих ключей
Плохо перемешивающая функция распределения	равномерная хэш-функция по ключу; консистентное хэширование с виртуальными узлами, чтобы смена топологии не перемешивала ключи (*Karger et al., 1997*)
Случайная маршрутизация между **взаимозаменяемыми** узлами	выбор из двух случайных узлов, запрос получает тот, у кого короче очередь (*Mitzenmacher, 2001*)
Долгоживущие соединения за L4-балансировщиком	балансировка по запросам (L7), ограничение времени жизни соединения
Тяжёлые запросы блокируют лёгкие	изоляция по классам, раздельные пулы
Хвост задержки при большом числе узлов	для идемпотентных запросов: дубль на реплику, если ответа нет дольше ~p_{95}, берётся первый ответ; без паузы нагрузка удваивается (*Dean, Barroso, 2013*)

- Среднюю загрузку кластера **смотреть недостаточно**: нужны метрики **по каждому** шарду и экземпляру
- Максимум по узлам важнее среднего: система отказывает на самом нагруженном узле
- Выбор из двух узлов не лечит горячий **ключ**: запрос обязан попасть на «свой» шард

3.4 Запас мощности

Запас (headroom) — плата за **стабильную задержку** и за **неопределённость**.

В модели $M/M/1$ (лекция 2) среднее время в системе (ожидание плюс обслуживание): $W = \dfrac{S}{1-\rho}$. Оно распределено экспоненциально, поэтому $p_{99} = W \ln 100 \approx 4.6\,W$.

Загрузка ρ	0.5	0.6	0.7	0.8	0.9
W/S	2.0	2.5	3.3	5.0	10
p_{99}/S	9.2	11.5	15.4	23.0	46.1

Два разных запаса, их нельзя путать:

- Ёмкость μ — **максимум RPS на экземпляр, при котором SLO по хвосту ещё выполняется** (нагрузочный тест), а не точка насыщения. Первый запас, «до насыщения», уже заложен в μ
- Целевая загрузка ρ_{target} — доля от μ. Это **второй, дополнительный** запас на ошибки оценки: профиль нагрузки, рост трафика, деградация зависимостей
- $M/M/1$ — осторожная оценка: у экземпляра с c воркерами ($M/M/c$, формула Эрланга C) очередь при той же загрузке короче

3.4 Сколько нужно экземпляров

$$n \geq \frac{\lambda_{peak}}{\mu \cdot \rho_{target}}$$

, $L = \lambda W$ (закон Литтла) даёт **среднее** число запросов в системе.

Пример: $\lambda_{peak} = 12\,000$ RPS, $\mu = 500$ RPS на экземпляр при выполнении SLO, $\rho_{target} = 0.6$, средняя задержка $W = 50$ мс.

Расчёт	Результат
Экземпляров при $\rho_{target} = 0.6$	$12000/(500 \cdot 0.6) =$ **40**
Запросов в системе	$L = 12000 \cdot 0.05 =$ **600**
В среднем на экземпляр ($n = 40$)	$600/40 =$ **15**
Слотов пула, чтобы реализовать μ	$k \geq \mu W = 500 \cdot 0.05 =$ **25** (загрузка пула $15/25 = 0.6$)

- **15 — не размер пула**, а среднее число запросов в системе. Пул из k слотов пропускает не больше k/W: при $k = 15$ это 300 RPS, ровно средняя нагрузка на экземпляр, то есть пул загружен на 100%
- Закон Литтла работает со **средним** W, а не с p_{99}. Число слотов — это μW, где W — время, которое запрос занимает слот
- Не каждый запрос держит соединение с БД: пул соединений считаем так же, по времени удержания соединения. Суммарно $n \cdot k$ соединений ограничено возможностями БД (см. 3.3)

3.4 Потеря части мощности

Мощность, которую вы оплачиваете, не равна мощности, которая **доступна** в худший момент:

Источник потери	Что происходит
Отказ узла или зоны доступности	пропадает доля экземпляров
Обновление (rolling update)	часть экземпляров недоступна или прогревается
Прогрев после запуска	кэши и JIT пусты, экземпляр обслуживает меньше
Соседи, GC, троттлинг	ёмкость экземпляра временно ниже измеренной

Пример: $z = 3$ зоны, после потери одной оставшиеся $\frac{z-1}{z} = \frac{2}{3}$ мощности обязаны выдержать пик с $\rho \le 0.8$ от μ:

$n \ge \frac{12000}{500 \cdot 0.8} \cdot \frac{3}{2} = 45$ экземпляров (по 15 в зоне). В норме $\rho = 12000/(45 \cdot 500) \approx 0.53$.

В аварии мы **тратим часть второго запаса**: SLO ещё выполняется ($\rho < 1$ от μ), но запас на неопределённость падает с 40% до 20%.

Условие «при потере зоны» оказалось строже (45 против 40): **размер парка определяет худший сценарий**.

3.4 Худший сценарий: зона и обновление

Rolling update тоже уменьшает доступную мощность.
В Kubernetes параметры `maxUnavailable` и `maxSurge` задают, сколько экземпляров можно остановить и сколько лишних запустить на время обновления.

Пусть при обновлении одновременно недоступно до 25% экземпляров, и это совпало с потерей зоны на пике: $n \geq \frac{12000}{500 \cdot 0.8} \cdot \frac{1}{\frac{2}{3} \cdot \frac{3}{4}} = 30 \cdot 2 = 60$ экземпляров (по 20 в зоне)

Сценарий	Парк n
Пик, $\rho_{target} = 0.6$	40
Пик и потеря зоны	45
Пик, потеря зоны и обновление с `maxUnavailable=25%`	60

- Переход от 45 к 60 стоит ещё трети парка: **совпадение трёх событий — решение о риске**, а не формула
- Дешевле: обновлять с `maxSurge` (новые экземпляры поднимаются до остановки старых), не обновлять в пик, ограничивать скорость обновления
- Принятый сценарий и его цену фиксируем расчётом

3.4 Пик и автомасштабирование

- Автомасштабирование **реагирует с задержкой**: сбор метрик, решение, запуск экземпляра, прогрев. Если всплеск короче этой цепочки, помогает только заранее заложенный запас
- Для ожидаемых пиков (суточный цикл, акции) — масштабирование **по расписанию**, для непредсказуемых — по метрике с запасом
- У сервисов, упирающихся не в CPU, масштабируем по метрике, **связанной с насыщением** (очередь, in-flight); у CPU-bound сервисов годится и CPU. Но задержка и in-flight растут и при сбое зависимости ниже по цепочке: тогда масштабирование лишь усилит нагрузку на неё
- Автомасштабирование не лечит **узкое место ниже по цепочке** (3.3): оно лишь переведёт нагрузку на БД
- Обязательны **верхняя граница** числа экземпляров и защита зависимостей от лавины запросов (лимиты, circuit breaker)

Масштабирование под пик — это экономика: запас стоит денег, нехватка — SLO.

Выбирайте осознанно и фиксируйте решение расчётом.

Чек-лист инженера

1. Прежде чем масштабировать, **оптимизируйте и измерьте** один экземпляр
2. Проверьте, что сервис **допускает** несколько экземпляров, а состояние вынесено
3. Оцените α и β по замерам $X(N)$, **не** экстраполируйте линейно
4. Найдите **узкое место** по загрузке и насыщению, масштабируйте именно его
5. После каждого изменения **ищите следующее** ограничение
6. Следите за метриками **по каждому** шарду и экземпляру, а не только за средним
7. Считайте n **по худшему сценарию**: пик плюс потеря зоны плюс обновление

Заключение

Идея	Суть
Виды масштабирования	scale up проще, но с потолком; scale out требует допускающей это архитектуры
Состояние	stateless масштабируется легко, сложность уходит в stateful-слой
Закон Амдала	$S(\infty) = 1/(1-p)$: последовательная доля задаёт потолок
USL	α — конкуренция, β — согласование; при $\beta > 0$ есть пик N^*
Узкое место	$X_{max} = \min_i \mu_i / V_i$; после масштабирования смещается
Горячий ресурс	кластер упирается в самый нагруженный узел
Мощность	$n \geq \lambda_{peak} / (\mu \rho_{target})$: μ — ёмкость при выполнении SLO, ρ_{target} — дополнительный запас; худший сценарий — потеря зоны и обновление

Следующая лекция: что ломается при выходе за пределы одного узла, то есть **фундаментальные ограничения распределённых систем**.

Список источников

- Amdahl. *Validity of the single processor approach to achieving large scale computing capabilities* (AFIPS Spring Joint Computer Conference, 1967)
- Gustafson. *Reevaluating Amdahl's Law* (Communications of the ACM, 1988)
- Gunther. *Guerrilla Capacity Planning* (Springer, 2007)
- McSherry, Isard, Murray. *Scalability! But at what COST?* (HotOS, 2015)
- Dean, Barroso. *The Tail at Scale* (Communications of the ACM, 2013)
- Kleppmann. *Designing Data-Intensive Applications* (O'Reilly, 2017): масштабируемость, секционирование и перекос нагрузки
- Beyer, Jones, Petoff, Murphy (ред.). *Site Reliability Engineering* (O'Reilly, 2016)
- Lazowska, Zahorjan, Graham, Sevcik. *Quantitative System Performance* (1984); Denning, Buzen (ACM Computing Surveys, 1978)
- Karger et al. *Consistent Hashing and Random Trees* (STOC, 1997)
- Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing* (IEEE TPDS, 2001)
- Harchol-Balter. *Performance Modeling and Design of Computer Systems* (Cambridge, 2013)
- Kubernetes Documentation. *Deployments: Rolling Update Deployment* (`maxUnavailable`, `maxSurge`)

Обратная связь

Анонимная форма